agronomy

MDPI

Article

DS-YOLO: A Lightweight Strawberry Fruit Detection Algorithm

Hao Teng [1], Fuchun Sun [2,*], Haorong Wu [1], Dong Lv [3], Qiurong Lv [2], Fan Feng [1], Sichen Yang [3] and Xiaoxiao Li [1]

[1] School of Electronic Information and Electrical Engineering, Chengdu University, Chengdu 610106, China; wuhaorong3150@cdu.edu.cn (H.W.); lixiaoxiao@cdu.edu.cn (X.L.)
[2] School of Mechanical Engineering, Chengdu University, Chengdu 610106, China; 212023085501014@cdu.edu.cn
[3] Chengdu Institute of Metrology Verification and Testing, Chengdu 610000, China
* Correspondence: sfc@cdu.edu.cn; Tel.: +86-199-8040-9129

Abstract

Strawberry detection in complex orchard environments remains a challenging task due to frequent leaf occlusion, fruit overlap, and illumination variability. To address these challenges, this study presents an improved lightweight detection framework, DS-YOLO, based on YOLOv8n. First, the backbone network of YOLOv8n is replaced with the lightweight StarNet to reduce the number of parameters while preserving the model's feature representation capability. Second, the Conv and C2f modules in the Neck section are replaced with SlimNeck's GSConv (hybrid convolution module) and VoVGSCSP (cross-stage partial network) modules, which effectively enhance detection performance and reduce computational burden. Finally, the original CIoU loss function is substituted with WIoUv3 to improve bounding box regression accuracy and overall detection performance. To validate the effectiveness of the proposed improvements, comparative experiments were conducted with six mainstream object detection models, four backbone networks, and five different loss functions. Experimental results demonstrate that the DS-YOLO achieves a 1.7 percentage point increase in mAP50, a 1.5 percentage point improvement in recall, and precision improvement of 1.3 percentage points. In terms of computational efficiency, the number of parameters is reduced from 3.2M to 1.8M, and computational cost decreases from 8.1G to 4.9G, corresponding to reductions of 43% and 40%, respectively. The improved DS-YOLO model enables real-time and accurate detection of strawberry fruits in complex environments with a more compact network architecture, providing valuable technical support for automated strawberry detection and lightweight deployment.

Keywords: smart agriculture; strawberry detection; YOLOv8; StarNet

Academic Editor: Marco Fontanelli

Received: 14 August 2025
Revised: 11 September 2025
Accepted: 19 September 2025
Published: 20 September 2025

Citation: Teng, H.; Sun, F.; Wu, H.; Lv, D.; Lv, Q.; Feng, F.; Yang, S.; Li, X. DS-YOLO: A Lightweight Strawberry Fruit Detection Algorithm. *Agronomy* **2025**, *15*, 2226. https://doi.org/10.3390/agronomy15092226

1. Introduction

Strawberries, as a widely cultivated economic fruit, are rich in vitamin C, anthocyanins, and various minerals, offering significant nutritional and economic value [1]. However, the strawberry harvesting process faces considerable challenges, which severely impact the sustainable development of the industry. Currently, strawberry picking is heavily reliant on manual labor, resulting in problems like high labor intensity, insufficient efficiency, tight harvesting windows, and a shortage of agricultural workers [2]. Due to the thin skin and fragility of strawberries, as well as the need for timely harvesting, manual methods struggle to complete the task efficiently, limiting industry profitability and expansion. Additionally, the low-lying, sprawling growth habit of strawberry plants, coupled with densely clustered fruits often obscured by lush leaves, further complicates mechanical harvesting. While agricultural robotics has been gradually emerging, current detection

methods mainly require manually incorporated variables like fruit texture, color, and shape for identification [3]. These approaches are often influenced by detrimental elements like lighting variations, leaf obstruction, and background noise in natural complex scenes, leading to significant reductions in detection performance, which fails to meet practical application needs. Furthermore, in such environments, robots require high-precision visual algorithms for efficient and non-damaging harvesting. Reckless actions without accurate identification can easily result in fruit damage or mechanical failure. Therefore, developing a strawberry fruit detection model with high accuracy, strong occlusion resistance, and lightweight deployment on mobile harvesting platforms is crucial to overcoming the bottlenecks in strawberry industry automation and promoting industry upgrades.

In recent years, as traditional agriculture shifts towards automation, substantial advancements have been achieved in object detection algorithms for fruit identification. Mainstream detection algorithms are usually divided into two types: one involves region proposal-based two-stage detection algorithms [4], such as RCNN (Regions with Convolutional Neural Network) [5] and Fast RCNN [6–8]; the other category is single-stage regression-based detection algorithms, including SSD [9] and the YOLO [10–13] series. Two-stage algorithms excel in detection accuracy by reducing background interference through candidate box filtering. However, the serial processing in two stages results in slower inference speeds and higher computational demands, making them inappropriate for immediate harvesting. Conversely, single-stage algorithms, with their end-to-end architecture, eliminate the candidate box generation step, offering faster detection and simpler workflows, making them better suited for deployment on mobile platforms. Consequently, research in fruit target recognition has gradually shifted towards using single-stage models. In corn-related detection tasks, Lawal [14] applied dilated convolutions to YOLOv3 for tomato detection, achieving 98.3% accuracy, but the model complexity hindered lightweight deployment. Tang et al. [15] developed a YOLOv4-tiny-based algorithm for oil-tea camellia fruit, improving feature extraction while retaining high speed, though accuracy remained limited. Gai et al. [16] constructed YOLO-V4-dense, which enhanced accuracy by 15% through deeper feature extraction but significantly increased computational demands. Yang et al. [17] integrated SPD-Conv into YOLOv5 for corn disease detection, improving spatial feature representation but with limited generalizability. Chen et al. [18] improved YOLOv7 for maize ear detection, achieving a 12.2% accuracy gain, though lightweight efficiency was constrained. For citrus detection, Lin et al. [19] proposed AG-YOLO with Next-ViT as the backbone to address citrus misdetections, but Transformer-based structures substantially increased computational load. Lv et al. [20] optimized YOLOv7 for citrus detection with higher accuracy and reduced complexity, yet detection in dense environments remained problematic. Sun et al. [21] presented YOLOv7-MCSF for grape detection under lighting and occlusion challenges, though its improvements were crop-specific. In other categories of fruit detection, Nan et al. [22] introduced WGB-YOLO based on YOLOv3 for dragon fruit, achieving robustness across scenarios, but at the expense of model size. Chen [23] developed FEW-YOLO based on YOLOv8 for goji berry detection, improving accuracy by 1.89% and reducing inference time, though the gains were modest. Hou et al. [24] proposed YOLOv8-CML for color-changing melon detection, achieving high precision while maintaining lightweight design, yet performance in highly complex orchard environments was still limited.

From these studies, several key insights emerge. First, lightweight models offer advantages in parameter reduction and inference speed but often suffer performance drops in dense, occluded, and lighting-variant orchard environments, indicating insufficient robustness. Second, high-precision models achieve strong results under controlled conditions but impose excessive computational costs, limiting deployment on resource-constrained

agricultural robots. Third, despite many YOLO variants applied to fruit detection, specific research targeting strawberries remains relatively limited. The unique characteristics of strawberries—low plant height, clustered growth, and severe leaf occlusion—make achieving both lightweight and accurate detection particularly challenging.

Therefore, this study aims to address the unique challenges of strawberry detection in complex orchard environments by balancing lightweight deployment with high accuracy and robustness. Building upon existing lightweight detection models, we propose a novel approach that incorporates a lightweight backbone to reduce model size, an efficient feature fusion structure to enhance multi-scale representation, and an advanced bounding box regression loss to improve localization accuracy. Unlike prior studies that typically focus on single structural modifications or task-specific crops, our work emphasizes integrating multiple strategies to jointly improve performance under occlusion, overlapping fruits, and variable illumination.

This research proposes a lightweight method for strawberry detection to address the issues, DS-YOLO, which is an improved version of YOLOv8n, aimed at meeting the requirements of mechanized strawberry harvesting. The primary contributions of this paper are summarized as follows.

1. We employ the lightweight backbone StarNet to substitute the original backbone of YOLOv8n. By utilizing its element-wise multiplication operation, the model is able to extract features more efficiently while reducing parameter complexity.
2. We integrate SlimNeck by replacing the Conv and C2f modules in the Neck with GSConv and VoVGSCSP. This adjustment helps balance detection accuracy with computational cost, improving efficiency in resource-constrained environments.
3. We adopt WIoUv3 as the bounding box regression loss instead of CIoU, which provides more stable optimization and improves localization accuracy, particularly in challenging orchard scenes.

2. Materials and Methods

2.1. Dataset Collection

Our research developed a bespoke dataset to precisely represent the appearance characteristics of strawberries in natural orchard conditions. The data acquisition was conducted on strawberries grown in a typical greenhouse cultivation area in Chengdu. The region uses a traditional greenhouse planting model, where raised beds are used for planting in double rows. This method allows strawberry fruits to hang down onto plastic film, with the roots spaced relatively far apart, providing ample space for the fruits in all directions. This planting model created favorable conditions for data collection. The image resolution is 4284×5712, ensuring the quality of the dataset.

During the data collection phase, to comprehensively cover the various growth stages, positions, and lighting conditions of the strawberries, multi-dimensional shooting was performed. The operator moved to different locations within the greenhouse and captured the strawberry plants from multiple angles (e.g., frontal, side, and top-down views), ensuring that all sides of the fruit and their relative positions to the leaves and background were recorded. A total of 800 images were captured, which include scenes of single fruits, multiple fruits, and fruits partially occluded, providing a rich set of samples for subsequent model training. Figure 1 shows strawberry fruit samples captured under various conditions in complex environments.

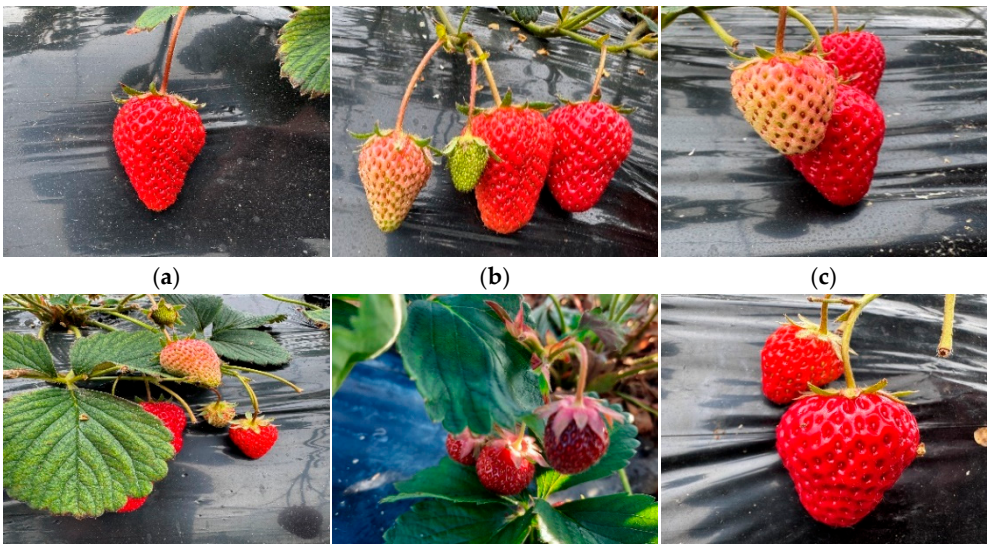


Figure 1. Strawberry fruit samples: (**a**) single target; (**b**) multiple targets; (**c**) fruit overlap; (**d**) leaf occlusion; (**e**) sunny day; (**f**) cloudy day.

2.2. Data Preprocessing

To strengthen the model's resilience and its capacity to generalize, data augmentation techniques were used to expand our dataset [25]. Multiple strategies for expanding the dataset were employed, including horizontal flipping, brightness and contrast adjustment, and the addition of random noise. For example, some images were horizontally flipped to simulate the effect of capturing from different directions; brightness and contrast were adjusted to simulate different lighting conditions; Gaussian noise was added to improve the model's resilience to noise interference. Simultaneously, application probability for each augmentation method was set reasonably based on the specific circumstances to ensure that the augmented dataset maintained diversity without excessively altering the original image features. Figure 2 shows images processed with different data augmentation methods. With data augmentation applied, the dataset was extended to 1920 images. The dataset distribution and the number of strawberry instances are shown in Table 1, which includes various fruit states, lighting conditions, and shooting angles, providing more sufficient data support for model training. The dataset was partitioned into training, validation, and testing subsets in a ratio of 8:1:1 to ensure a balanced evaluation of the model's performance.

Table 1. Dataset distribution and the number of strawberry instances.

Category	Number of Images	Number of Strawberry Samples
Original image	800	3071
Image flip	240	956
Random noise	240	1015
Adjust brightness	200	932
Image sharpening	240	1071
Fog effect	200	879
Total	1920	7924

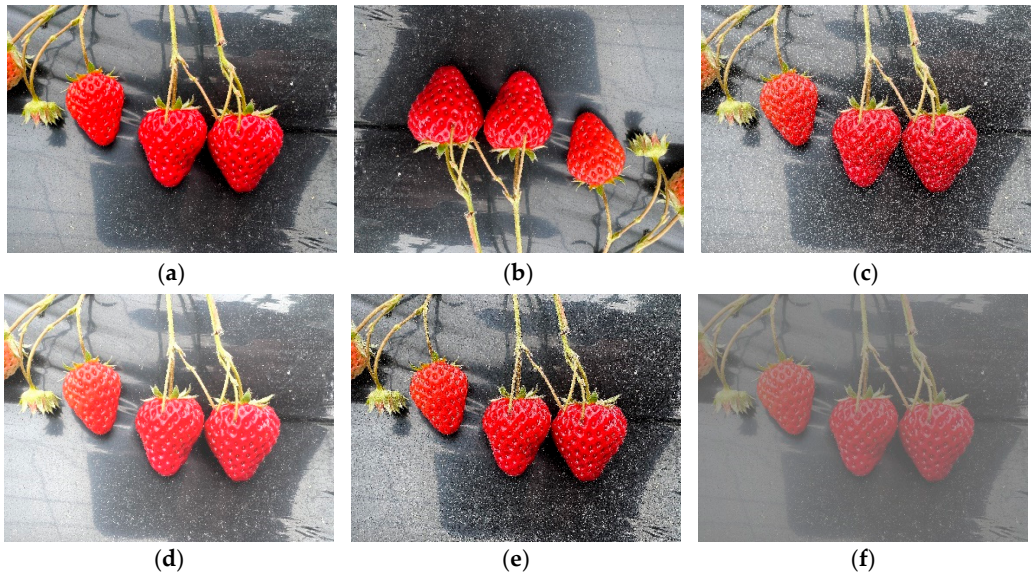


Figure 2. Augmented strawberry fruit samples: (**a**) original image; (**b**) image flip; (**c**) random noise; (**d**) adjust brightness; (**e**) image sharpening; (**f**) fog effect.

2.3. YOLOv8 Algorithm Model

The YOLOv8 algorithm is a novel object detection and image segmentation integrated algorithm developed and launched by Ultralytics in January 2023, derived from YOLOv5. It exhibits strong capability and can run on various platforms, including CPUs and GPUs [26]. Figure 3 shows the network architecture.

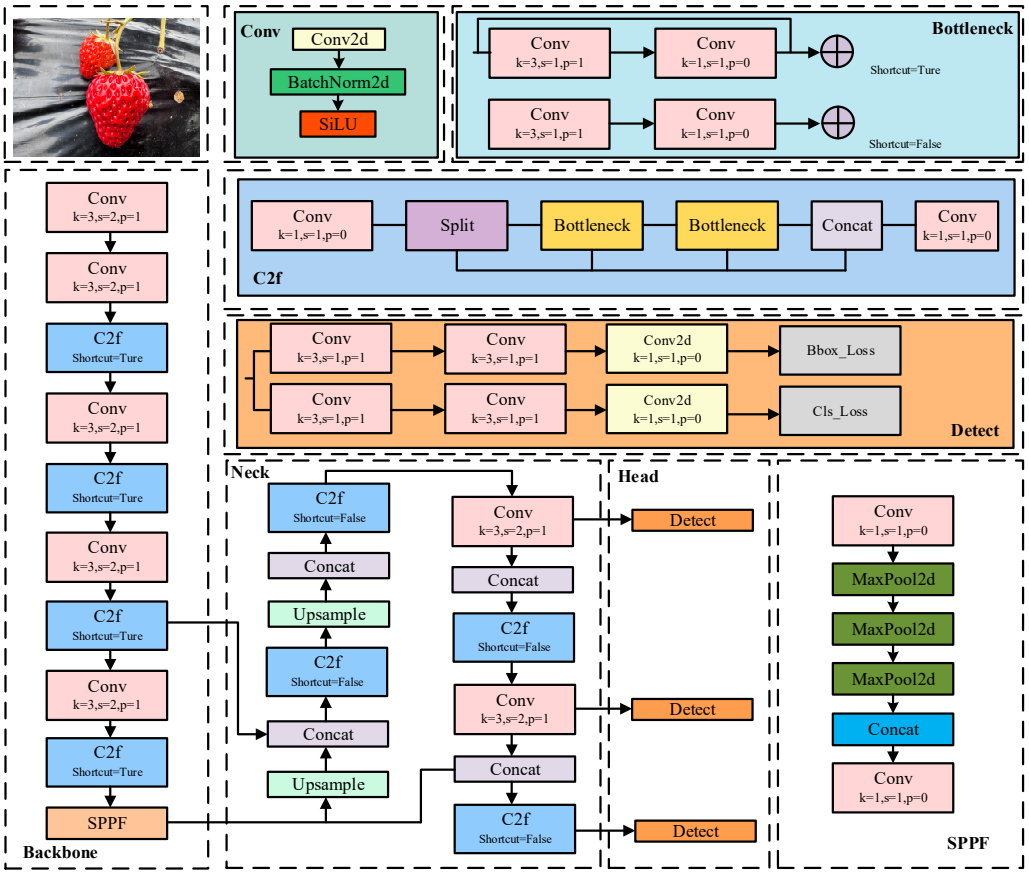


Figure 3. YOLOv8 Network Architecture Diagram.

YOLOv8, as an enhanced version of the single-stage object detection algorithm, significantly improves detection accuracy while maintaining the advantage of high-speed detection. The algorithm's network design comprises three components: the backbone network, the neck network, and the detecting head [27]. The backbone network uses an improved CSPDarknet [28] structure, which divides the feature map into two parts using the Cross-Stage Partial (CSP) module. One part performs deep convolutions to extract high-level semantic features, while the other part connects across stages to retain low-level detail information. This design reduces computation and effectively mitigates the vanishing gradient problem, allowing the network to learn layered features that capture target information at multiple scales from the input image. The neck network utilizes an enhanced Path Aggregation Network (PAN) [29], employing bidirectional path aggregation—both top-down and bottom-up—to execute upsampling or downsampling operations on maps produced by the backbone. Once the spatial dimensions are unified, convolution is used for feature fusion, integrating shallow detail features with deep semantic features, thus providing the detection head with richer multi-scale contextual information. The detection head has independent classification and regression heads to handle different tasks. The classification head outputs the probability distribution of the predicted bounding boxes' class, while the regression head is responsible for predicting the target's location coordinates (center coordinates, width, and height). This decoupling design avoids task interference and enhances the flexibility of model optimization.

In the training phase, YOLOv8 incorporates several key technologies to enhance generalization and training efficiency: For data augmentation, the Mosaic data augmentation technique is employed, which randomly stitches together four images to create new samples, increasing data diversity and simulating complex scenes. The adaptive anchor box generation strategy dynamically generates anchor boxes derived from the dimensional distribution of entities inside the dataset, making model more suited to the specific task's target shape. The learning rate scheduling uses the cosine annealing algorithm [30], dynamically adjusting the learning rate to avoid oscillations in the later stages of training, speeding up model convergence and improving stability.

Overall, YOLOv8 achieves efficient fusion of multi-scale features and task decoupling through an improved network architecture. It enhances detection accuracy through dynamic sample matching and fine-grained loss function optimization, strengthens the model's generalization ability with data augmentation and adaptive training strategies, and, while maintaining the speed advantage of one-stage algorithms, provides an optimized performance balance for object detection tasks.

2.4. DS-YOLO

To resolve the concerns related to large model parameter sizes and the difficulty in efficiently recognizing strawberry fruits in complex environments in current strawberry recognition methods, the DS-YOLO detection model was designed by optimizing the original YOLOv8n network model. This modification seeks to fulfill the deployment criteria for Entry-level equipment. The precise enhancements are enumerated as follows:

1. Lightweight network StarNet is used to replace the backbone of YOLOv8n, reducing the number of parameters while maintaining the model's feature representation capabilities.
2. The Conv and C2f modules in the Neck section are replaced with the GSConv module and the VoVGSCSP module from SlimNeck, effectively enhancing object detection performance and alleviating the computational burden.
3. WIoUv3 substitutes the original CIoU loss function in YOLOv8n for loss computation, enhancing the model's bounding box regression efficacy and detection precision.

Figure 4 illustrates the configuration of the enhanced network model.

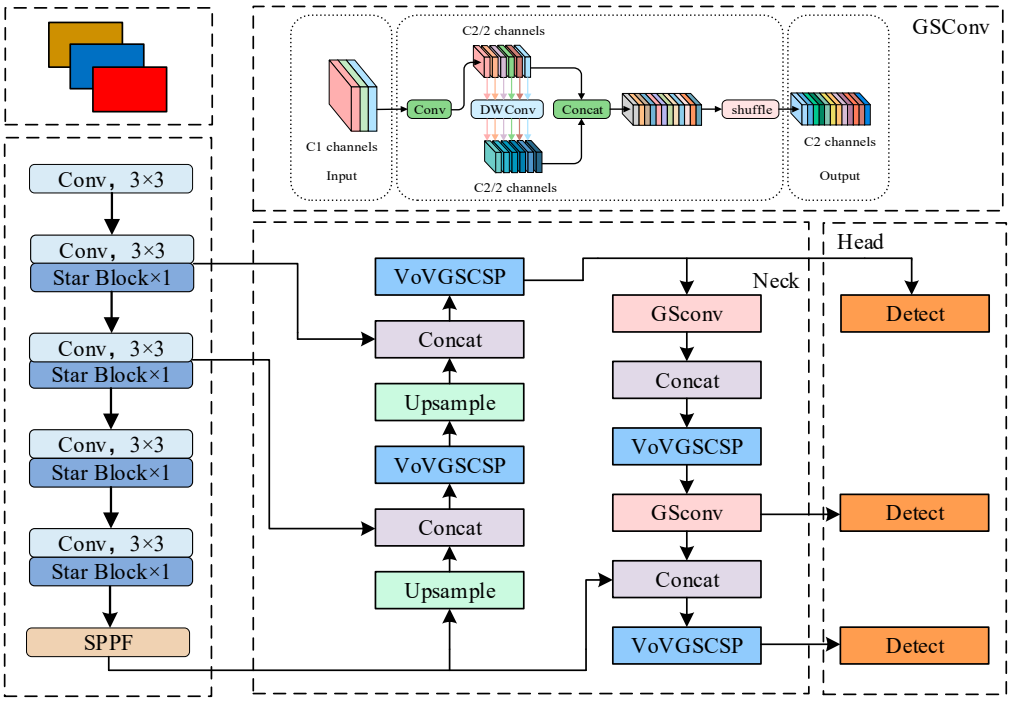


Figure 4. The network structure of DS-YOLO.

2.4.1. StarNet Network

The fundamental YOLOv8 model commonly employs CSPDarknet53 as its backbone network. It employs the Cross Stage Partial (CSP) architecture, which improves feature representation by partitioning the feature map into two segments and processing them through distinct pathways. This structure also includes multiple down-sampling layers, making it suitable for capturing target features at different scales, and it performs well for detecting large objects. Although this design improves performance, it inevitably leads to an increase in parameter count and computational complexity. Additionally, traditional convolution operations have limited capability in modeling spatial information, particularly when dealing with small objects or complex scenes, which may result in the loss of edge and detail features. Therefore, this study introduces the efficient and lightweight StarNet [31] network as a replacement for part of the original backbone network. This network, designed by Microsoft in 2024, offers high efficiency and light weight, and the structure diagram is Figure 5.

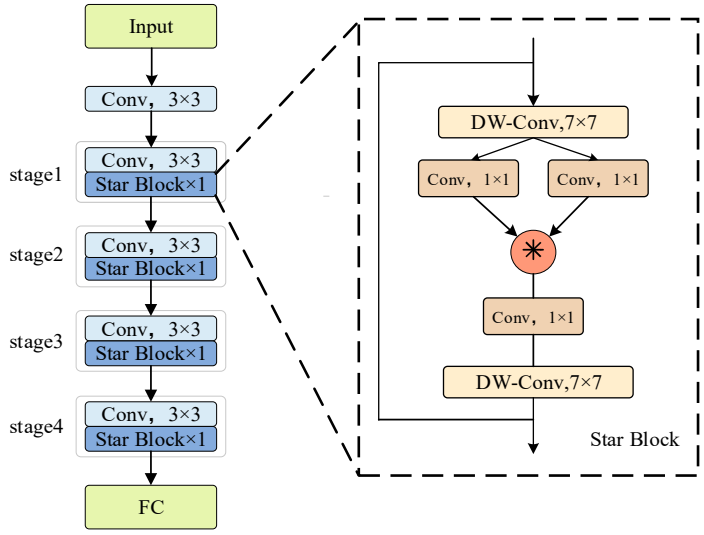


Figure 5. The structure of StarNet.

This network is a lightweight convolution module that combines spatial attention mechanisms with residual structures. By decomposing traditional convolution operations and introducing spatial attention, it enhances the model's capability to discern spatial characteristics about target. The StarNet network adopts a four-level hierarchical structure. At each level, the first operation is convolution for down-sampling, which decreases the dimensions of the feature map and performs initial feature extraction. Then, the Star Block [32] is applied for further processing. Input features were first extracted through a DW-Conv [33], followed by two independent 1×1 convolutions for dimensionality expansion. The output features are then fused through a unique element-wise multiplication operation, enhancing feature interaction and fusion capabilities. Finally, high-level features are further extracted through the depthwise convolution (DW-Conv), followed by feature dimensionality reduction, and the results are output.

The central component of StarNet is the star-shaped computation mechanism. In a single-layer network, the star-shaped operation fuses two linearly transformed features through element-wise multiplication. The expression is typically written as wT1x $\times$ wT2x, where W = [W B]$^{\mathrm{T}}$ and X = [X 1]$^{\mathrm{T}}$. Expanding the formula gives the equation shown in Equation (1), where d represents the input channel number, i and j are the channel indices, and α is the coefficient for each term.

$$\begin{aligned} w_1^T x * w_2^T x &= \left(\begin{array}{c} i=1 \\ \sum\limits_{d+1}^{i=1} w_1^i x^i \end{array} \right) * \left(\begin{array}{c} j=1 \\ \sum\limits_{d+1}^{j} w_2^j x^j \end{array} \right) \\ &= \sum_{i=1}^{d+1 d+1} w_1^i w_2^j x^i x^j \\ &= \alpha_{(1,1)} x^1 x^1 + \cdots + \alpha_{(4,5)} x^4 x^5 + \cdots + \alpha_{(d+1,d+1)} x^{d+1} x^{d+1} \end{aligned} \tag{1}$$

Through the above equation, a single channel can be expanded into multiple channels and multiple feature elements can be processed simultaneously. These technique improves the model's ability to handle different feature interactions and enables more efficient representation of spatial relationships in the data. In which, $W_1, W_2 \in R^{(d+1)\times(d'+1)}$, $X \in R^{(d+1)\times n}$, like Equation (2).

$$\alpha_{(i,j)} = \begin{cases} w_1^i w_2^j & if i = j \\ w_1^i w_2^j + w_1^j w_2^i & if i! = j \end{cases} \tag{2}$$

The process in Equation (2) generates a total of (d + 2)(d + 1)/2 distinct terms. Except for the $\alpha_{(d+1,;)} x^{(d+1)} x$ term, the remaining terms represent nonlinear combinations of the input features, effectively expanding the implicit dimensionality of the input features without adding additional computational burden. Therefore, let the spatial dimension be t; performing the star operation in this space allows for the expression of features in a $\frac{(d+2)(d+1)}{2} \approx \left(\frac{d}{\sqrt{2}}\right)^2$ dimensional space, significantly enlarging the feature dimensions and augmenting the model's capacity to identify in intricate situations like as blurriness, occlusion, or low contrast.

2.4.2. Slim-Neck NetWork

The Neck component in the YOLOv8 model integrates multi-scale characteristics to augment feature representation efficacy. To further improve capability, we replaced the original Neck with Slim-Neck [34], a lightweight module designed to optimize the neck part of convolutional networks. The main structure of Slim-Neck consists of three components: GSConv, GS Bottleneck, and VoV-GSCSP.

2.4.3. GSConv

GSConv [35] serves as the fundamental element of the Slim-Neck module, comprising standard convolution (SC), depthwise separable convolution (DSC), and shuffle operations. While DSConv efficiently diminishes the computational complexity, its architecture leads to the segregation of channel information, potentially compromising the integrity of feature extraction. Conversely, ordinary convolution exhibits superior feature extraction capabilities, but at a higher computational expense. GSConv was developed as a lightweight convolution that integrates regular convolution with depthwise separable convolution to resolve these concerns. The technique involves partitioning the feature map into many groups along the channel dimension by ordinary convolution, followed by the application of depthwise separable convolution to each group. This method guarantees precision while diminishing computational complexity. Figure 6 illustrates the architecture of the GSConv module.

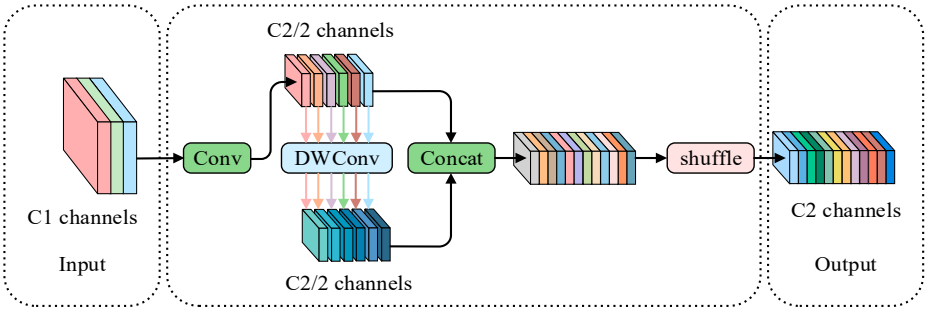


Figure 6. GSConv architecture.

The input feature map first passes through a convolution layer, with the output channels set to C/2. It then undergoes a depthwise separable convolution (DWConv) layer, which performs depthwise separable convolution on each channel of the input image individually. After that, the outputs of the convolution layer and the DWConv are connected. Finally, the data underwent random shuffle to rearrange the feature channels, improving the flow of information. Finally, the result is passed through C2 output channels for the final output.

Typically, we use FLOPs to define the complexity of convolutional computations. The time complexity formulas for standard convolution and GSConv are as follows:

$$FLOPs_{SC} = w \times h \times k \times k \times 1 \times c', \tag{3}$$

$$FLOPs_{GSConv} = w \times h \times k \times k \times \frac{c'}{2} \times (c+1), \tag{4}$$

2.4.4. GS Bottleneck & VoV-GSCSP

Fundamental elements of the GS Bottleneck [36] module comprise an input feature map processing sequence and the feature fusion method. The main structure consists of two serially connected GSConv (Group Spatial Convolution) layers, both of which employ grouped convolution techniques. These layers enhance feature representation capabilities by parallelizing feature processing. Additionally, a skip connection (using addition) is designed to fuse the outputs of the two GSConv layers with original input. This connection method preserves the initial feature information while promoting efficient information flow within the bottleneck structure through feature aggregation. After processing with two layers of GSConv and merging through the skip connection, the final output is an optimized and integrated feature map, achieving a balance between fine-grained feature capture and computational efficiency. The structure is presented in Figure 7a.

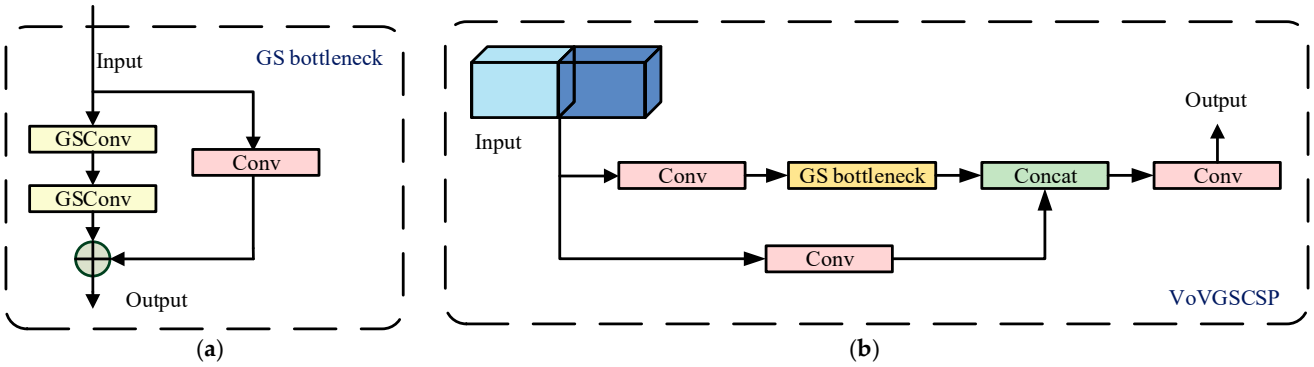


Figure 7. Structure of GSbottleneck and VoVGSCSP. (**a**) GSbottleneck; (**b**) VoVGSCSP.

VoVGSCSP section employs a singular aggregation technique to construct a cross-stage partial network module that enhances the GS Bottleneck module. It is used to effectively fuse information between feature maps at different stages. Figure 7b presents the structure of this model. Starting with a standard convolution layer, it is responsible for performing initial feature extraction on the input feature map, generating basic features. These are then processed by a GS Bottleneck unit, which contains two serially connected GSConv layers. The features processed by the GS Bottleneck are then concatenated with the output from the initial convolution layer, achieving a fusion of low-level feature convolution outputs with high-level features, maintaining both feature richness and detail preservation. Finally, another convolution layer integrates and optimizes the concatenated fusion features, producing the final output feature map.

This design minimizes computing complexity and network architecture complexity while preserving enough accuracy. It facilitates efficient information transmission and integration across feature maps at various stages, enabling the network to refine the usage of multi-scale features, hence enhancing target detection performance.

2.5. WIoUv3 Loss Function

In complex orchard environments, strawberry detection tasks often involve a significant number of small object detections. Therefore, designing a more reasonable loss function can significantly improve the model's detection ability. In YOLOv8, the CIoU [37] loss is commonly used for calculating the boundary box loss, as shown in the diagram below. Despite enhancing the regression accuracy of the bounding box, this function still possesses the following limitations:

1. CIoU penalizes the difference in aspect ratio between the predicted and ground truth boxes to optimize shape. However, the penalty is only based on a fixed formula, which may not be adaptable to all scenarios.
2. CIoU relies on IoU (Intersection over Union) as the core component, and the gradient calculation may cause numerical fluctuations. When the overlap between the predicted box and ground truth is very low (IoU ≈ 0), the CIoU gradient may become unstable.
3. CIoU uses a uniform loss computation for all targets, which may not be suitable for optimizing the detection of tiny targets. The boundary box of these targets itself has a smaller area, and fluctuations in CIoU have a more significant impact on the loss. Additionally, the estimation error in aspect ratio may be more prominent.

The calculation formula is as follows:

$$L_{\mathrm{CIoU}} = 1 - IoU + \frac{\rho^2(b, b^{\mathrm{gt}})}{c_{\mathrm{w}}^2 + c_{\mathrm{h}}^2} + \frac{4}{\pi^2}\left(\tan^{-1}\frac{w_{gt}}{h_{gt}} - \tan^{-1}\frac{w}{h}\right) \quad (5)$$

Some parameter definitions involved in Equation (6) are shown in Figure 8. IoU used to evaluate the degree of overlap between the predicted bounding box and the ground truth. ρ^2 (b, b^{gt}) is the Euclidean distance between the centroids of the ground truth box and the predicted box. h and w denote the height and width of the predicted boundary box, respectively; h_{gt}, w_{gt} represent the height and width of the ground truth boundary box. c_h, c_w represent the height and width of the minimum enclosing rectangle formed by the predicted and ground truth boundary boxes.

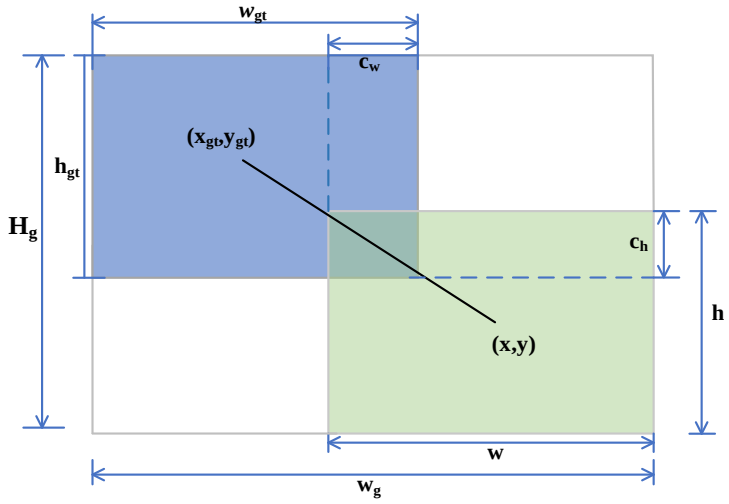


Figure 8. CIoU Diagram.

In the practical application of YOLOv8, there are many typical loss functions besides CIoU. EIoU (Enhanced IoU) decomposes the aspect ratio of CIoU to avoid the coupling issue of the aspect ratio penalty term. However, its modeling of the aspect ratio is still not flexible enough and may not be suitable for targets with irregular shapes. SIoU (Shape IoU) introduces direction-awareness (angle cost) and shape matching (shape cost) on top of CIoU, further optimizing the rotation and shape of the bounding box. This makes SIoU more effective in dense and rotated object detection. GIoU (Generalized IoU) calculates the penalty term by introducing the minimum enclosing box to IoU, addressing the gradient problem when there is no overlap.

Although these four typical loss functions improve model performance in different ways, WIoU not only addresses issues like aspect ratio but also introduces a dynamic weighting mechanism to bypass the limits of typical loss functions in terms of object scale, shape, and sample distribution. Currently, WIoU has evolved into its third generation. WIoUv1, as the initial version, introduces scale and shape weights to adaptively adjust the loss contributions of different targets, mitigating gradient instability when IoU ≈ 0 through a gradient smoothing mechanism. WIoUv2, based on the first generation, introduces direction-awareness weights to address the problem of rotating object detection and optimizes gradient propagation paths to improve training stability. In WIoUv3 [38], outlier value β is introduced to define anchor box quality, along with a β-based focusing factor r. The focusing factor is then incorporated into the base WIoUv1, and the calculation formula for WIoUv1 is as follows:

$$L_{\mathrm{WIoUv1}} = R_{\mathrm{WIoU}} \times L_{\mathrm{IoU}} \tag{6}$$

$$R_{\mathrm{WIoU}} = \exp\left(\frac{\left(b_{cx} - b_{gt}^{cx}\right)^2 + \left(b_{cy} - b_{gt}^{cy}\right)^2}{c_w^2 + c_h^2}\right) \tag{7}$$

$$L_{\mathrm{IoU}} = 1 - \mathrm{IoU} \tag{8}$$

As the value of β decreases, the quality of the anchor box increases, and the corresponding r value allocated in the overall loss function becomes smaller. This also indicates that when the value of β increases, the anchor box quality decreases, and the gradient gain becomes smaller. WIoUv3 adopts a unique gradient gain distribution method, which causes the model to prioritize the average quality of samples, thereby improving the overall performance of the model. Its calculation formula is as follows:

$$L_{\mathrm{WIoUv3}} = \left(1 - \frac{W_{\mathrm{i}} H_{\mathrm{i}}}{S_{\mathrm{u}}}\right) \exp\left[\frac{(x - x_{\mathrm{gt}})^2 + (y - y_{\mathrm{gt}})^2}{(W_{\mathrm{g}}^2 + H_{\mathrm{g}}^2)^*}\right] \gamma \tag{9}$$

$$\gamma = \frac{\beta}{\delta \alpha^{\beta - \delta}} \tag{10}$$

$$\beta = \frac{L_{\mathrm{IoU}}^*}{\overline{L}_{\mathrm{IoU}}} \in [0, +\infty) \tag{11}$$

In the formula, β represents the outlier degree of the anchor box, where a smaller outlier degree indicates a higher quality of the anchor box; γ is the non-monotonic focusing coefficient, which reduces the interference of low-quality anchor boxes during training; δ is a hyperparameter; (x,y) and (x_{gt},y_{gt}) are the center coordinates of the predicted box and the ground truth box, respectively; Wg and Hg are the width and height of the minimum enclosing region that contains both the ground truth box and the predicted box; Su denotes the area of the non-intersecting zone between the anticipated box and the ground truth box.; L^*_{IoU} is IoU loss for current anchor box; and the mean IoU loss is denoted by $\overline{L}_{\mathrm{IoU}}$.

3. Results

3.1. Experimental Platform Configuration and Training Strategy

The configuration of the experimental hardware and software platform for this study is presented in Table 2.

Table 2. Experimental environment hardware and software configuration parameters.

Component	Specification
CPU	13thGenIntel(R)Core(TM)i5-13400F
GPU	NVIDIA GeForce RTX 4060 Ti
OperatingSystem	Windows10
Python	3.8.20
Pytorch	2.0.1
CUDA	11.7
cuDNN	8.9.4

In the training phase, YOLOv8n.pt was chosen as the pre-trained weight, and a custom strawberry fruit dataset was utilized. The primary parameter configurations for model training are presented in Table 3.

Table 3. Training Parameters.

Training Parameters	Values
Input Image Size	640 × 640
Batch Processing Size	32
Training Batch Size	400
Initial Learning Rate	0.01
Learning Rate Decay Ratio	0.01
Weight Decay Coefficient	0.0005
Learning Rate Momentum Coefficient	0.937
Optimizer	SGD

3.2. Evaluation Metrics

In object detection tasks, different evaluation metrics assess the effectiveness of detection algorithms from multiple angles. The main metrics are: Precision (P), Recall (R), Mean Average Precision (mAP), the number of parameters (Param), and Floating Point Operations (FLOPs). The formulas for Precision, Recall, and Average Precision are as follows:

$$P = \frac{TP}{TP + FF} \tag{12}$$

$$R = \frac{TP}{TP + FN} \tag{13}$$

$$AP = \int_0^1 P(R)dR \tag{14}$$

$$mAP = \frac{1}{n}\int_1^n AP_i \tag{15}$$

In these formulas, TP denotes true positives, which are samples predicted as positive that are indeed positive; FP denotes false positives, which are samples projected as positive that are actually negative; FN denotes false negatives, referring to samples projected as negative that are, in fact, positive. APi signifies the Average Precision for class i, whereas n specifies the total number of classes.

3.3. Comparison Experiments

To validate the effectiveness of the algorithm proposed in this study, A comparison is conducted between the suggested method and many established classical object detection methods, Faster R-CNN, SSD, YOLOv3 [39], YOLOv5n [40], YOLOv6 [41], YOLOv7-tiny [42], YOLOv8n, YOLOv10n, YOLOv11n and YOLOv12. These models were trained and tested for comparison with the initial YOLOv8n model. Table 4 presents the evaluation metrics of eleven network models after training.

Table 4. Evaluation of Detection Efficacy Among Various Algorithmic Models.

Model	P%	R%	mAP50%	Parameters/106	FLOPs/G
Faster-R-CNN	83.6	81.2	86.1	97.3	360.9
SSD	86.5	75.9	73.1	80.6	76.4
YOLOv3	93.7	95.8	98.3	103.6	282.2
YOLOv5n	90.4	93.6	96.1	2.5	7.1
YOLOv6	92.5	94.1	95.4	4.2	11.8
YOLOv7-tiny	96.2	96.8	97.1	6.1	13.2
YOLOv8n	94.3	92.6	95.5	3.2	8.1
YOLOv10n	94.1	96.9	97.9	2.3	6.5
YOLOv11n	94.6	97.1	98.9	2.6	6.3
YOLOv12	91.9	98.1	98.5	2.5	6.3
OURS	95.6	94.1	97.9	1.8	4.9

From Table 4, it is evident that there are significant differences in both performance metrics (Precision P%, Recall R%, mAP50%) and efficiency parameters (parameter count, computational cost) among the different object detection models. Our model demonstrates outstanding performance in both aspects. Regarding detecting efficacy, the precision of our model reaches 95.6%, ranking second only to YOLOv7-tiny and significantly outperforming traditional models, as well as lightweight models like YOLOv3 and YOLOv5n. The recall rate is 94.1%, on par with YOLOv6, slightly lower than YOLOv3 and YOLOv7-tiny, but notably higher than SSD and Faster R-CNN, indicating strong target detection ability. The mAP50% of our model is 97.9%, slightly lower than YOLOv3, but higher than YOLOv7-tiny

and YOLOv5n, confirming that its overall detection accuracy is excellent. In comparative experiments with state-of-the-art baseline models, the algorithm proposed in this paper exhibits a slight deficiency in terms of recall and average precision when compared to YOLOv10, YOLOv11, and YOLOv12. Nevertheless, it demonstrates distinct advantages in both precision and computational complexity.

In terms of efficiency, our model outperforms all others, with a parameter count of only 1.8×10^6 and a computational cost of just 4.9G, far lower than the other models. As shown in Figure 9, the DS-YOLO achieves a balance between detection ability and lightweight design, demonstrating significant advantages over current detection algorithms.

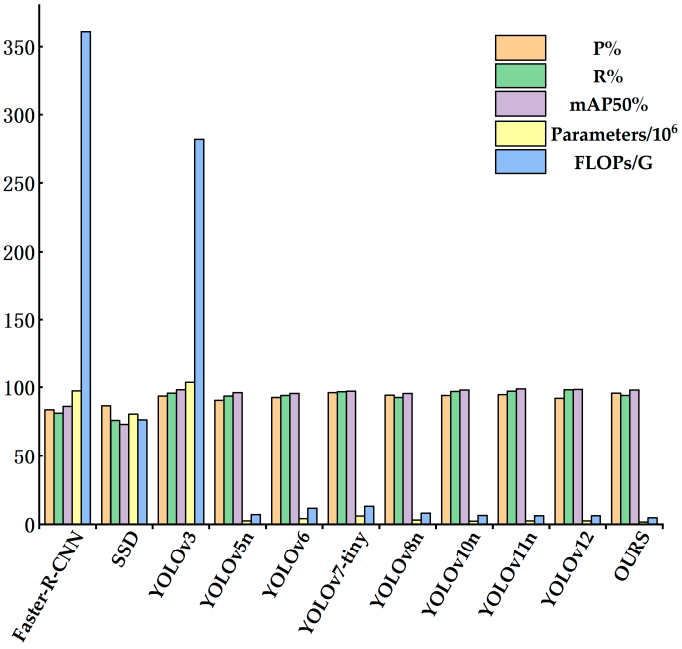


Figure 9. Columnar analysis of the mainstream models.

3.4. Ablation Experiment

An ablation experiment was undertaken to validate the practicality of the upgraded modules we proposed for strawberry fruit detection, utilizing YOLOv8n as the foundational network model for each optimization. Eight experimental groups were established, and the findings are presented in Table 5.

Table 5. Ablation experiment.

Groups	StarNet	SlimNeck	WIoUv3	P/%	R/%	mAP50/%	Params/106	FLOPs/G
1	×	×	×	94.3	92.6	95.5	3.21	8.1
2	√	×	×	94.7	93.5	96.2	1.98	5.7
3	×	√	×	94.6	91.8	95.9	2.81	7.4
4	×	×	√	94.8	92.8	95.6	3.21	8.1
5	√	√	×	95.2	93.7	96.9	1.77	4.9
6	√	×	√	94.9	92.5	96.4	1.97	5.7
7	×	√	√	94.7	93.2	95.8	2.8	7.4
8	√	√	√	95.6	94.1	97.1	1.81	4.9

Note: √ indicates that the module is used; × indicates that the module is not used.

Based on the experimental results in Table 5, it can be observed that Group 1, as the baseline YOLOv8n model, achieved a precision of 94.3%, recall of 92.6%, and mAP50 of 95.5%, with Params of 3.21×10^6 and FLOPs of 8.1G. In Group 2, where only StarNet was used, the model's performance showed consistent improvement: precision increased

by 0.4 percentage points, recall rose by 0.9 percentage points, and mAP50 improved by 0.7 percentage points, while the parameter count decreased by 38.3%, and the computational load reduced by 30.9%. This demonstrates that its adaptive feature aggregation mechanism enhances multi-scale feature correlation while improving computational efficiency. When only SlimNeck was used (Group 3), mAP50 increased by just 0.4%, but recall decreased by 0.8 percentage points, with the parameter count reduced by 12.5%. This trade-off indicates that while the lightweight neck enhances model compactness, it can impair low-level feature retention without complementary modules. When only WIoUv3 was enabled (Group 4), the performance change was negligible, indicating that this loss function has limited impact on baseline features, and its localization optimization capabilities depend on high-quality feature representations.

In the module combination experiments, the synergistic effects were significantly evident. The combination of StarNet and SlimNeck (Group 5) achieved an mAP50 of 96.9% and Params is1.77×10^6, indicating that SlimNeck's slimming effect on the redundant feature maps generated by StarNet was more pronounced. After adding WIoUv3 (Group 8), mAP50 improved to 97.1%, precision reached 95.5%, recall was 94.1%, while the parameter count remained at 1.81×10^6, and the computational load was 4.9G. This three-component fusion formed a closed-loop optimization mechanism: StarNet enhanced feature discriminability, SlimNeck reduced redundant information, and WIoUv3 achieved fine localization on the optimized features.

It is noteworthy that the performance of Groups 6 and 7 was inferior to that of Groups 5 and 8, which verifies the logic that feature enhancement and structural optimization should precede the fine-tuning of loss functions.

3.5. Comparison of Backbone Networks in Experiments

The YOLOv8n backbone network, when performing feature extraction, faces issues such as poor small-object localization and insufficient feature extraction, leading to inefficiency in small-object detection tasks. Since strawberries are small fruits, to improve the model's localization capability for strawberries in different environments, a lightweight StarNet network substitutes the previous backbone. A comparison was made between StarNet and four mainstream backbone networks and the results present in Table 6.

Table 6. Comparison Results of Backbone Networks.

Backbone	P/%	R/%	mAP50/%	Parameters/106	FLOPs/G
MobileNetV3	93.6	94.2	95.1	2.78	6.9
ShuffleNet	94.5	94.7	95.8	2.93	7.2
GhostNetV2	94.2	95.1	94.7	2.65	6.8
FasterNet	95.1	94.8	94.6	2.41	6.3
EfficientViT	95.6	94.3	95.3	2.27	6.1
StarNet	94.7	93.5	96.2	1.98	5.7

As shown in the results of Table 6, the StarNet backbone network we used reaches a notably higher mAP50 in comparison with MobileNetV3, ShuffleNet, GhostNetV2, and FasterNet. Although FasterNet records the highest precision at 95.1%, its mAP50 is 1.6% lower than that of StarNet, indicating a relatively weaker capacity for discriminating complex visual patterns. While GhostNetV2 demonstrates a relatively high recall rate, its poor balance between precision and recall results in the lowest mAP50 among the compared models.

In terms of computational efficiency, StarNet exhibits clear advantages over other typical backbone networks. It requires only 1.98×10^6 parameters and 5.7 GFLOPs, which are 28.4% and 17.4% lower, respectively, than those of the next most efficient model,

FasterNet. Additionally, StarNet maintains comparable precision and recall to ShuffleNet and FasterNet, achieving lightweight design without compromising detection performance.

In summary, the StarNet backbone network effectively reduces computational resource demands while maintaining high detection accuracy for strawberry fruits, thereby meeting the requirements for fruit recognition in complex environments.

3.6. Loss Function Comparison

To assess the efficacy of various loss functions, the CIoU in YOLOv8n was individually substituted by EIoU, SIoU, WIoUv1, WIoUv2 and WIoUv3. Comparative experiments were conducted under identical conditions. The results of these loss function comparisons are presented in Figure 10.

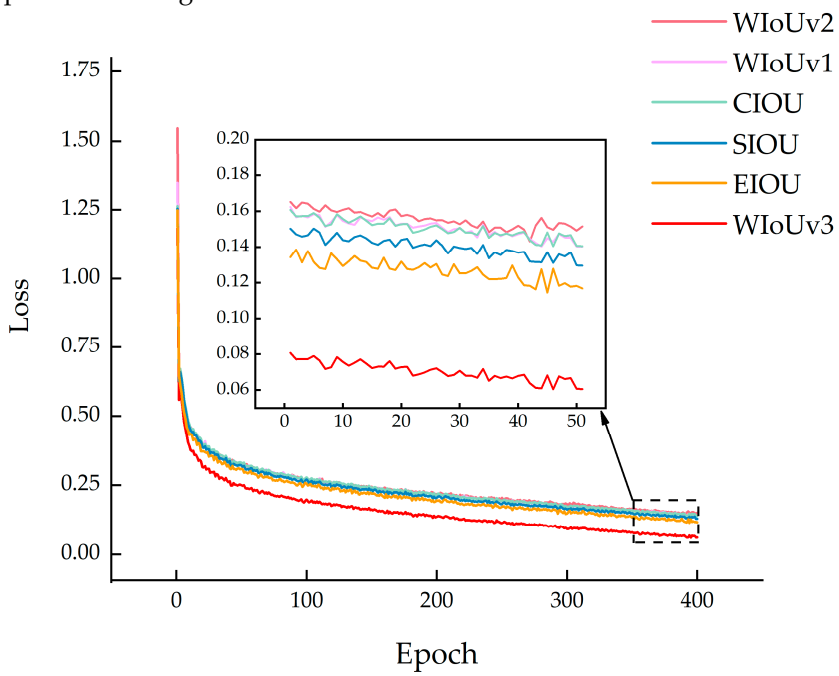


Figure 10. Comparison Experiment of Loss Functions.

3.7. Comparative Experiments of the Improved DS-YOLOv8

To validate the superiority of the DS-YOLO strawberry detection network in identifying strawberry targets under orchard conditions, a comparative experiment was conducted between YOLOv8 and DS-YOLO on a strawberry dataset under consistent environmental conditions and training strategies, ensuring the objectivity of the results. According to Figure 11, DS-YOLO demonstrates a significant performance advantage: compared with YOLOv8n, its precision for citrus fruit recognition improved by 0.3%, recall increased by 4%, and the mAP metric rose by 3.5%, indicating enhanced detection accuracy in complex orchard scenarios.

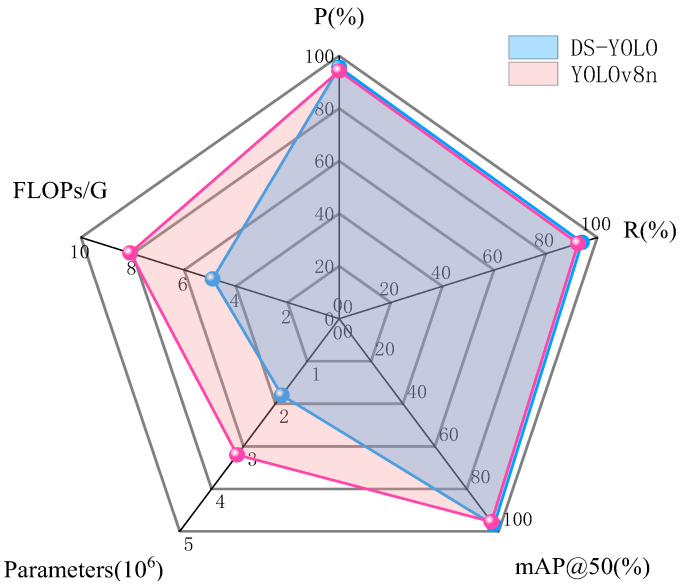


Figure 11. Comparative Experiment Between the Improved DS-YOLOv8 and YOLOv8n.

Furthermore, through architectural optimization of the baseline model, DS-YOLO achieved a 15.4% reduction in the Parames and achieved a 23.7% decrease in computational complexity (GFLOPs). These improvements not only reduce the demand for computational hardware requirements but also accelerate processing speed, thereby ensuring efficient target detection of strawberry fruits in challenging environments. This provides a significant advantage for the practical application of machine vision technologies in agricultural settings.

3.7.1. Detection Result Visualization

To demonstrate the advantages of the DS-YOLO in detection tasks, pictures from different scenarios were selected for comparative experiments, and the visualized bounding box results are shown in Table 7.

Table 7. Detection Results in Different Scenarios.

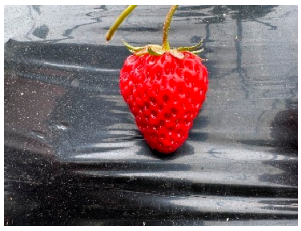
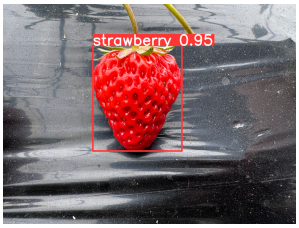
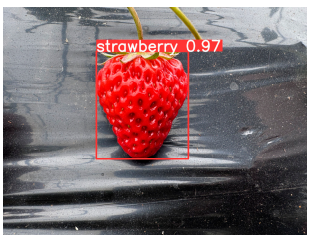
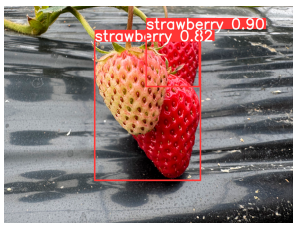
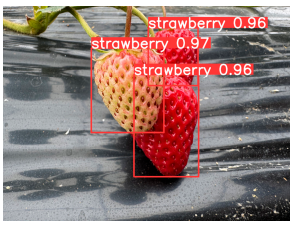
Category	Original Image	YOLOv8	DS-YOLOv8
Single Target			
Multiple Targets			

Table 7. *Cont.*

Category	Original Image	YOLOv8	DS-YOLOv8
Strong Light			
Low Light			
Leaf occlusion			
Fruit overlap			

3.7.2. Visualized Heatmap Analysis

To more clearly illustrate the efficacy of the suggested enhancements, this study employs the Grad-CAM [43] technique for visual analysis, generating heatmaps of the results, which are presented in Table 8. In these heatmaps, deeper red regions indicate higher model attention and a greater probability of target presence, whereas deeper blue regions represent lower attention and a lower likelihood of the target being present.

Table 8. Strawberry Fruit Detection Heatmaps under Different Scenarios.

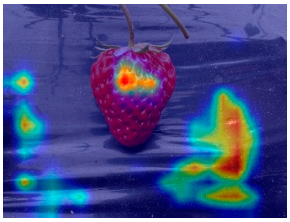
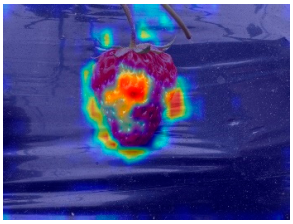
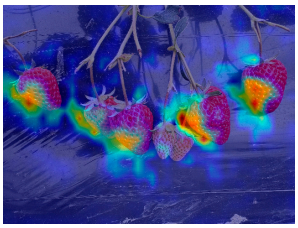
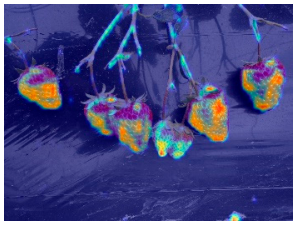
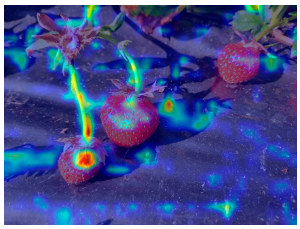
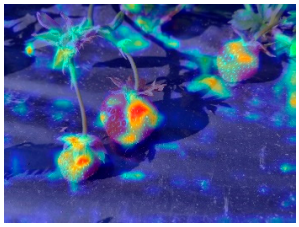
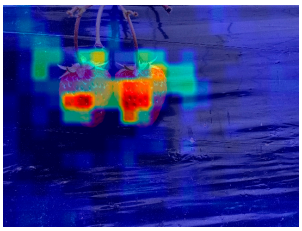
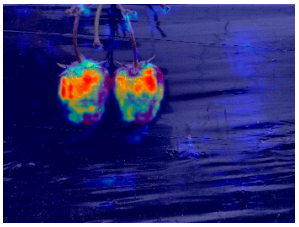
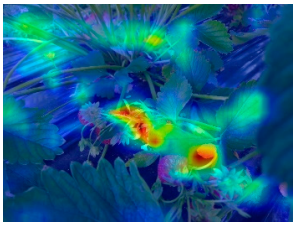
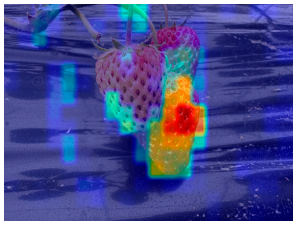
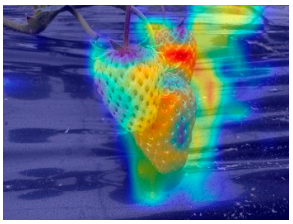
Category	Original Image	YOLOv8n	DS-YOLO
Single Target			

Table 8. *Cont.*

Category	Original Image	YOLOv8n	DS-YOLO
Multiple Targets			
Strong Light			
Low Light			
Leaf occlusion			
Fruit overlap			

As observed in Table 8, compared with YOLOv8n, the heatmaps generated by the DS-YOLO model more accurately align with the actual regions of strawberry fruits, emphasizing the extraction of edge features more effectively. Under varying environmental conditions, the DS-YOLO heatmaps consistently outperform those of the original YOLOv8n, thereby validating the improved algorithm's superior detection performance and robustness when applied to strawberry fruits in complex orchard scenarios.

4. Discussion

This study introduces a lightweight strawberry recognition system, DS-YOLO, which is an improvement of the YOLOv8n baseline model. The principal enhancements encompass the incorporation of the StarNet backbone network, the SlimNeck neck architecture (comprising GSConv and VoVGSCSP), and the WIoUv3 loss function. The element-wise multiplication operations in StarNet augment the model's capacity to differentiate features of occluded fruits, whereas the dynamic gradient allocation mechanism in WIoUv3 en-

hances bounding box regression performance in densely clustered environments, achieving a balance between precision and computational efficiency.

Ablation studies and comparative experiments with several representative backbone networks and loss functions validate the effectiveness and advancement of the proposed modules. Furthermore, comparison with the YOLOv8n baseline model under various scenarios demonstrates that DS-YOLO performs more robustly in complex environments, making it more suitable for real-world applications.

Despite the advancements achieved, two major limitations remain that warrant further investigation.

First, the current model lacks validation under actual deployment conditions. While theoretical metrics such as parameter count and FLOPs suggest that DS-YOLO is suitable for deployment on edge devices, empirical testing on real-world hardware platforms like Jetson Nano and Raspberry Pi has yet to be conducted. Key practical indicators—including inference latency, energy consumption, and memory usage—are essential for confirming the model's viability in real-time and resource-constrained agricultural settings. To address this, future work should involve rigorous deployment tests across a range of embedded hardware platforms, evaluating performance under real-world greenhouse and field conditions. In addition, building diverse, scenario-specific datasets tailored for different deployment configurations (such as stationary greenhouse arms or mobile platforms in open orchards) will enable more targeted training, improving the model's stability in various situations. In robotic harvesting systems, this limitation directly affects operational reliability and feasibility. If inference latency is too high, grasping points may become outdated before the manipulator executes the motion, leading to failed picking attempts. Similarly, excessive energy consumption can shorten the robot's working time in greenhouse environments, while high memory usage may limit the integration of other essential functions such as navigation, obstacle avoidance, or path planning. Without rigorous hardware validation, even a lightweight and theoretically efficient model may fail to deliver stable and cost-effective performance in practice.

Second, our model does not distinguish between different stages of fruit maturity. In practical harvesting, the ripeness of strawberries—commonly indicated by a color shift from white to red—is a crucial factor in decision-making. However, the system currently treats all detected fruits uniformly, regardless of their maturity level. This limitation restricts its applicability in precision harvesting scenarios where selective picking is required. Future studies could incorporate multi-class classification schemes or integrate color and texture cues to categorize strawberries into various ripeness stages (e.g., unripe, partially ripe, fully ripe). Moreover, the use of temporal image sequences or multimodal inputs (such as RGB-NIR fusion) could further enhance the system's maturity estimation capabilities. Equipping the model with this functionality would not only improve its decision-making intelligence but also significantly contribute to the advancement of smart agriculture by enabling automated, quality-driven harvesting. For robotic harvesting systems, the absence of maturity discrimination can cause several practical issues. The robot may pick immature strawberries, thereby reducing market value, or leave fully ripe fruits uncollected, resulting in economic losses. Moreover, without ripeness information, the robot cannot prioritize its harvesting sequence efficiently, which may reduce overall throughput and lead to suboptimal use of limited operation time in greenhouses. Integrating maturity classification is thus not only a technical enhancement but also a practical necessity for ensuring both product quality and harvesting efficiency.

In summary, DS-YOLO demonstrates strong potential in lightweight and accurate fruit detection. However, to ensure practical applicability and support broader adoption in precision agriculture, further optimization in hardware deployment and functional refinement is required.

5. Conclusions

To improve the detection efficacy of strawberry fruit targets in natural environments, this study presents improvements based on the YOLOv8n model. Specifically, the backbone layer is replaced with the efficient and lightweight StarNet module, and the original C2f and Conv are substituted with the GSConv lightweight convolution module and the VoVGSCSP module in Neck. Furthermore, the original CIoU loss function is replaced with WIoUv3 to improve detection efficiency and accuracy under complex environmental conditions. The findings can be summarized as follows:

1. The improved lightweight model, DS-YOLO, achieves a precision of 95.6%, recall of 94.1%, mean average precision (mAP) of 97.2%, floating point operations (FLOPs) of 4.9G, and a parameter count of 1.8M. Compared with the original YOLOv8n model, this represents increases of 1.3%, 1.5%, and 1.6% in P, R, and mAP, respectively, while reducing FLOPs and parameters by 39.5% and 43.7%. Compared to mainstream models, the enhanced model attains an optimal equilibrium between detection accuracy and model conciseness., demonstrating a significant performance advantage.
2. Comparative experiments further confirm that the proposed DS-YOLO model outperforms other mainstream models in the task of strawberry fruit detection. By optimizing the backbone, neck structure, and loss function, the proposed algorithm enhances feature extraction and detection performance. It achieves model lightweighting while improving detection accuracy, effectively reducing missed detections and maintaining stable performance in challenging scenarios such as varying lighting conditions and target occlusion. The Grad-CAM-based heatmap visualizations conducted under different environmental conditions also show better performance than the original YOLOv8n model, indicating strong robustness and validating the advantage of the proposed model in strawberry fruit detection.

In conclusion, this study explores the development of lightweight detection models for agricultural applications. The results indicate that the use of a lightweight backbone network, efficient feature fusion structures, and an improved loss function can enhance detection performance under challenging environmental conditions and facilitate deployment on edge devices with limited computational resources. Furthermore, future research should not only focus on fruit maturity recognition and multi-class fruit detection, but also explore deep integration with robotic arms, real-time navigation systems, and multi-sensor data fusion to enable fully automated harvesting systems. In addition, validating the model's adaptability and robustness across different cultivation modes and large-scale commercial farms is critical for practical applications. In the long term, these advancements are expected to accelerate the industrialization of smart agriculture, reduce dependence on manual labor, and promote sustainable and efficient production in open-field cultivation.

Author Contributions: Conceptualization, H.T. and F.S.; methodology, H.T. and F.S.; software, H.T.; validation, D.L. and H.W.; formal analysis, X.L.; investigation, Q.L. and F.F.; resources, X.L.; data curation, H.T.; writing—original draft preparation, H.T.; writing—review and editing, H.T., S.Y. and H.W.; funding acquisition, F.S. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Data Availability Statement: The dataset supporting this research is available within the article. Data access requests should be addressed to the corresponding author.

Conflicts of Interest: The authors declare no conflicts of interest.

References

1. Tang, Z.; Wu, Y.; Xu, X. The study of recognizing ripe strawberries based on the improved YOLOv7-Tiny model. *Vis. Comput.* **2024**, *41*, 3155–3171. [CrossRef]
2. Wu, F.; Guan, Z.; Garcia-Nazariega, M. Comparison of Labor Costs between Florida and Mexican Strawberry Industries: FE1023, 12/2017. *EDIS* **2018**, *2018*, FE1023. [CrossRef]
3. Fan, P.; Lang, G.; Guo, P.; Liu, Z.; Yang, F.; Yan, B.; Lei, X. Multi-feature patch-based segmentation technique in the gray-centered RGB color space for improved apple target recognition. *Agriculture* **2021**, *11*, 273. [CrossRef]
4. Nygaard, M.; Frederiksen, L.; Kjeldsen, A.M.; Koca, M. Performance Comparison of Single-Stage and Two-Stage Detection Models for Real-Time Traffic Applications. In Proceedings of the International Conference on Science, Engineering Management and Information Technology, Ankara, Turkey, 12–13 September 2024; Springer: Cham, Switzerland, 2025; pp. 398–411.
5. Turan, M.; Almalioglu, Y.; Araujo, H.; Konukoglu, E.; Sitti, M. Deep EndoVO: A recurrent convolutional neural network (RCNN) based visual odometry approach for endoscopic capsule robots. *Neurocomputing* **2018**, *275*, 1861–1870. [CrossRef]
6. Zhao, Q.; Liu, Y.J. Design of apple recognition model based on improved deep learning object detection framework Faster-RCNN. *Adv. Contin. Discret. Models* **2024**, *2024*, 49. [CrossRef]
7. Girshick, R. Fast R-CNN. In Proceedings of the 2015 IEEE International Conference on Computer Vision, Santiago, Chile, 7–13 December 2015; pp. 1440–1448.
8. Tong, X.; Liang, Z.; Qin, M.; Liu, F.; Yang, J.; Xiao, H.; Dai, W. DAM-Faster RCNN: Few-shot defect detection method for wood based on dual attention mechanism. *Sci. Rep.* **2025**, *15*, 22860. [CrossRef]
9. Wang, H.; Qian, H.; Feng, S.; Wang, W. L-SSD: Lightweight SSD target detection based on depth-separable convolution. *J. Real-Time Image Process.* **2024**, *21*, 33. [CrossRef]
10. Redmon, J.; Farhadi, A. YOLO9000: Better, faster, stronger. In Proceedings of the 2017 IEEE Conference on Computer Vision and Pattern Recognition, Honolulu, HI, USA, 21–26 July 2017; pp. 7263–7271.
11. Redmon, J.; Farhadi, A. YOLOv3: An incremental improvement. *arXiv* **2018**, arXiv:1804.02767. [CrossRef]
12. Bochkovskiy, A.; Wang, C.-Y.; Liao, H.-Y.M. YOLOv4: Optimal speed and accuracy of object detection. *arXiv* **2020**, arXiv:2004.10934. [CrossRef]
13. Redmon, J.; Divvala, S.; Girshick, R.; Farhadi, A. You only look once: Unified, real-time object detection. In Proceedings of the 2016 IEEE Conference on Computer Vision and Pattern Recognition, Las Vegas, NV, USA, 27–30 June 2016; pp. 779–788.
14. Lawal, M.O. Tomato detection based on modified YOLOv3 framework. *Sci. Rep.* **2021**, *11*, 1447. [CrossRef]
15. Tang, Y.; Zhou, H.; Wang, H.; Zhang, Y. Fruit detection and positioning technology for a *Camellia oleifera* C. Abel orchard based on improved YOLOv4-tiny model and binocular stereo vision. *Expert Syst. Appl.* **2023**, *211*, 118573. [CrossRef]
16. Gai, R.; Chen, N.; Yuan, H. A detection algorithm for cherry fruits based on the improved YOLO-v4 model. *Neural Comput. Appl.* **2023**, *35*, 13895–13906. [CrossRef]
17. Yang, H.; Sheng, S.; Jiang, F.; Zhang, T.; Wang, S.; Xiao, J.; Zhang, H.; Peng, C.; Wang, Q. YOLO-SDW: A method for detecting infection in corn leaves. *Energy Rep.* **2024**, *12*, 6102–6111. [CrossRef]
18. Chen, D.; Zhao, H.; Li, Y.; Zhang, Z.; Zhang, K. PMF-YOLOv8: Enhanced ship detection model in remote sensing images. *Inf. Technol. Control* **2024**, *53*, 1204–1220. [CrossRef]
19. Lin, Y.; Huang, Z.; Liang, Y.; Liu, Y.; Jiang, W. AG-YOLO: A rapid citrus fruit detection algorithm with global context fusion. *Agriculture* **2024**, *14*, 114. [CrossRef]
20. Lv, Q.; Sun, F.; Bian, Y.; Wu, H.; Li, X.; Li, X.; Zhou, J. A Lightweight Citrus Object Detection Method in Complex Environments. *Agriculture* **2025**, *15*, 1046. [CrossRef]
21. Sun, F.; Lv, Q.; Bian, Y.; He, R.; Lv, D.; Gao, L.; Wu, H.; Li, X. Grape Target Detection Method in Orchard Environment Based on Improved YOLOv7. *Agronomy* **2025**, *15*, 42. [CrossRef]
22. Nan, Y.; Zhang, H.; Zeng, Y.; Zheng, J.; Ge, Y. Intelligent detection of Multi-Class pitaya fruits in target picking row based on WGB-YOLO network. *Comput. Electron. Agric.* **2023**, *208*, 107780. [CrossRef]
23. Chen, Y.; Liu, Q.; Jiang, X.; Wei, Y.; Zhou, X.; Zhou, J.; Wang, F.; Yan, L.; Fan, S.; Xing, H. FEW-YOLO: A lightweight ripe fruit detection algorithm in wolfberry based on improved YOLOv8. *J. Food Meas. Charact.* **2025**, *19*, 4783–4795. [CrossRef]
24. Chen, G.; Hou, Y.; Cui, T.; Li, H.; Shangguan, F.; Cao, L. YOLOv8-CML: A lightweight target detection method for Color-changing melon ripening in intelligent agriculture. *Sci. Rep.* **2024**, *14*, 14400. [CrossRef]
25. Mumuni, A.; Mumuni, F. Data augmentation with automated machine learning: Approaches and performance comparison with classical data augmentation methods. *Knowl. Inf. Syst.* **2025**, *67*, 4035–4085. [CrossRef]
26. Zhang, Y.; Gao, G.; Chen, Y.; Yang, Z. ODD-YOLOv8: An algorithm for small object detection in UAV imagery. *J. Supercomput.* **2025**, *81*, 202. [CrossRef]
27. Deng, Q.; Du, L.; Han, W.; Ren, W.; Yu, R.; Luo, J. CB-YOLO: Composite dual backbone network for high-frequency transformer coding defect detection. *Signal Image Video Process.* **2024**, *18*, 5535–5548. [CrossRef]

28. Pan, B.; Xiang, J.; Zhang, N.; Pan, R. A fine-grained attributes recognition model for clothing based on improved the CSPDarknet and PAFPN network. *Signal Image Video Process.* **2025**, *19*, 230. [CrossRef]
29. Lei, Y.; Jin, K.; Qiu, Z.; Sun, Y.; Bai, H.; He, W. MPA-Det: Multi-path aggregation-based object detection framework for aerial visual computing. *Vis. Comput.* **2025**, *41*, 9395–9408. [CrossRef]
30. Li, Y.-X.; Wang, J.-S.; Zhang, S.-W.; Zhang, S.-H.; Guan, X.-Y.; Ma, X.-R. Arithmetic optimization algorithm with cosine transform-based two-dimensional composite chaotic mapping. *Soft Comput.* **2025**, *29*, 1289–1329. [CrossRef]
31. Dai, J.; Fu, L.; Li, Y.; Zhao, J.; Hanajima, N. Lightweight SL-YOLO algorithm for automotive fuel tank cover detection. *Signal Image Video Process.* **2025**, *19*, 444. [CrossRef]
32. Filippov, A.; Blokhin, A.; Golovina, M.; Kirila, T.; Kozina, N.; Rodchenko, S.; Tenkovtsev, A. Linear and star-shaped block copolymers of poly-2-alkyl-5,6-dihydrooxazines. Conformation of macromolecules and thermoresponsivity in water—Salt solutions. *Russ. Chem. Bull.* **2024**, *74*, 1838–1846. [CrossRef]
33. Zhanfang, Z.; Tuo, L. Enhancing wind turbine blade damage detection with YOLO-Wind. *Sci. Rep.* **2025**, *15*, 18667. [CrossRef] [PubMed]
34. Li, H.; Li, J.; Wei, H.; Liu, Z.; Zhan, Z.; Ren, Q. Slim-neck by GSConv: A lightweight-design for real-time detector architectures. *J. Real-Time Image Process.* **2024**, *21*, 62. [CrossRef]
35. Cai, S.; Zhang, X.; Mo, Y. A Lightweight underwater detector enhanced by Attention mechanism, GSConv and WIoU on YOLOv8. *Sci. Rep.* **2024**, *14*, 25797. [CrossRef]
36. Elhenidy, A.M.; Labib, L.M.; Haikal, A.Y.; Saafan, M.M. GY-YOLO: Ghost separable YOLO for pedestrian detection. *Neural Comput. Appl.* **2025**, *37*, 14907–14933. [CrossRef]
37. Wu, J.; Zhang, Y.; Shan, T.; Xing, Z.; Chen, J.; Guo, R. An additive feature fusion attention based on YOLO network for aircraft skin damage detection. *J. Supercomput.* **2025**, *81*, 627. [CrossRef]
38. Wang, J.; Zhang, L.; Peng, W.; Ma, S.; Zhu, C. Dynamic snake convolution enhanced YOLOv8s for hydraulic tunnel defect detection. *Signal Image Video Process.* **2025**, *19*, 674. [CrossRef]
39. Li, G.; Li, S.; Fang, X.; Luan, X.; Liu, F. An improved YOLOv3 model for detection of invasive *Saccharomyces cerevisiae* infections. *Multimed. Tools Appl.* **2024**, *84*, 14605–14622. [CrossRef]
40. Tian, J.-H.; Feng, X.-F.; Li, F.; Xian, Q.-L.; Jia, Z.-H.; Liu, J.-L. An improved YOLOv5n algorithm for detecting surface defects in industrial components. *Sci. Rep.* **2025**, *15*, 9756. [CrossRef] [PubMed]
41. Li, N.; Wang, M.; Yang, G.; Li, B.; Yuan, B.; Xu, S. DENS-YOLOv6: A small object detection model for garbage detection on water surface. *Multimed. Tools Appl.* **2024**, *83*, 55751–55771. [CrossRef]
42. Kumar, A.; Bhattacharjee, S.; Kumar, A.; Jayakody, D.N.K. Facial identity recognition using StyleGAN3 inversion and improved tiny YOLOv7 model. *Sci. Rep.* **2025**, *15*, 9102. [CrossRef]
43. Selvaraju, R.R.; Cogswell, M.; Das, A.; Vedantam, R.; Parikh, D.; Batra, D. Grad-CAM: Visual explanations from deep networks via gradient-based localization. In Proceedings of the 2017 IEEE International Conference on Computer Vision, Venice, Italy, 22–29 October 2017; pp. 618–626.